Assignment No: 4
Title of Assignment:

Implement a solution for a Constraint Satisfaction Problem using Branch and Bound / Backtracking for n-queens problem

1. Prerequisite:

Basic knowledge of CSP, Backtracking

2. Objective:

In this experiment, we will be able to do the following:

- Study how to place N queens on board with non attacking mode using backtracking.
- What is CSP Problem

3. Outcome: Successfully able to place N queens on board with non attacking mode using backtracking.

4. Software and Hardware Requirement:

Open Source C++ Programming tool like G++/GCC, python,java and Ubuntu.

5. Relevant Theory / Literature Survey:

Constraint Satisfaction Problem

CSP means solving a problem under certain constraints or rules

CSP depends on three components

X: It is a set of variables

D: It is a set of domains where the variables reside

There is a specific domain for each variable.

C: It is a set of constraints which are followed by the set set of variables

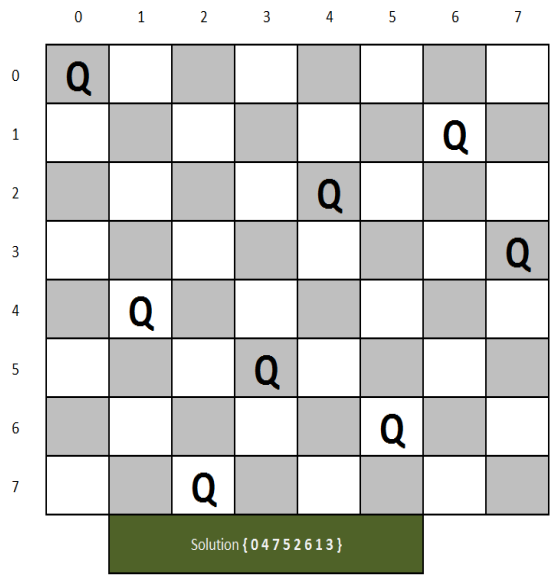
Backtracking Algorithm

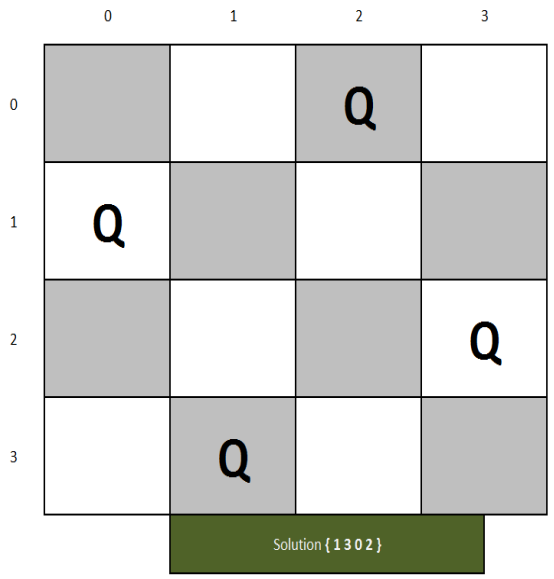
Backtracking is an algorithmic-technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point of time (by time, here, is referred to the time elapsed till reaching any level of the search tree).

N-Queens problem

The N-Queens problem is a puzzle of placing exactly N queens on an NxN chessboard, such that no two queens can attack each other in that configuration. Thus, no two queens can lie in the same row,column or diagonal.

The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes then we backtrack and return false.





N-Queens problem Algorithm

1) Start in the leftmost column

2) If all queens are placed return true

3) Try all rows in the current column.

Do the following for every tried row.

a) If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing the queen here leads to a solution.

b) If placing the queen in [row, column] leads to a solution then return true.

c) If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.

4) If all rows have been tried and nothing worked, return false to trigger backtracking.

Branch and Bound

Branch and bound is an algorithm design paradigm which is generally used for solving combinatorial optimization problems. These problems are typically exponential in terms of time complexity and may require exploring all possible permutations in the worst case. The Branch and Bound Algorithm technique solves these problems relatively quickly.

The branch and bound solution is somehow different, it generates a partial solution until it figures that there's no point going deeper as we would ultimately lead to a dead end.
In the backtracking approach, we maintain an 8x8 binary matrix for keeping track of safe cells (by eliminating the unsafe cells, those that are likely to be attacked) and update it each time we place a new queen. However, it required O(n^2) time to check the safe cell and update the queen.

Applying the branch and bound approach : The branch and bound approach suggests that we create a partial solution and use it to ascertain whether we need to continue in a particular direction or not.

Parameter	Backtracking	Branch and Bound
Approach	Backtracking is used to find all possible solutions available to a problem. When it realizes that it has made a bad choice, it undoes the last choice by backing it up. It searches the state space tree until it has found a solution for the problem.	Branch-and-Bound is used to solve optimization problems. When it realizes that it already has a better optimal solution that the pre-solution leads to, it abandons that pre-solution. It completely searches the state space tree to get an optimal solution.
Traversal	Backtracking traverses the state space tree by DFS(Depth First Search) manner.	Branch-and-Bound traverse the tree in any manner, DFS or BFS.
Function	Backtracking involves a feasibility function.	Branch-and-Bound involves a bounding function.
Problems	Backtracking is used for solving Decision Problems.	Branch-and-Bound is used for solving the optimization Problem.
Searching	In backtracking, the state space tree is searched until the solution is obtained.	In Branch-and-Bound as the optimum solution may be present anywhere in the state space tree, so the tree needs to be searched completely.
Efficiency	Backtracking is more efficient. in	Branch-and-Bound is less efficient.
Applications	Useful in solving N-Queen Problem, Sum of subset.	Useful solving Knapsack Problem, Traveling Salesman Problem.

6. Questions:

Q 1: Which are the constraints required to solve the N Queen Problem?

Q 2: Compare Backtracking and branch and bound methods.

Q 3: What do you mean by Constraint satisfaction problem?

7. Conclusion:

In This way we have studied how to solve the CSP problem and how to place N queens on board with non attacking mode using backtracking.